



Services in Kubernetes

Understanding Kubernetes for Orchestration

Introduction

Kubernetes **Pods** are temporary. When a Pod crashes or restarts, it gets a new IP address. If multiple Pods need to communicate, how do they find each other? This is where **Kubernetes Services** come into play.

A **Service** in Kubernetes provides a **stable way** to access and communicate with **Pods, Deployments, or external applications**.

This guide will cover:

- **What are Services?**
 - **Why do we need Services?**
 - **Types of Services in Kubernetes**
 - **How to create and manage Services**
 - **Troubleshooting and FAQs**
-

What is a Service in Kubernetes?

A **Service** is an abstraction that defines **how to access a group of Pods**.

- It assigns a **fixed IP address** and **DNS name** to a group of Pods.
- It allows communication between **Pods, Deployments, or external users**.
- It ensures **load balancing** and **fault tolerance**.

💡 *Think of a Service like a customer care number – even if representatives (Pods) change, the number (Service) stays the same!*



Why Do We Need Services?

- **Pods are temporary** – Services provide a stable endpoint.
- **Load balancing** – Services distribute traffic across multiple Pods.
- **External access** – Expose Kubernetes applications to users outside the cluster.
- **Inter-Pod communication** – Enable communication between microservices.

Without Services, Pods would be difficult to track and connect.

Types of Services in Kubernetes

Kubernetes provides different types of Services depending on the use case.

Service Type	Purpose
ClusterIP	Default type; exposes the Service only inside the cluster .
NodePort	Exposes the Service on a static port on each Node .
LoadBalancer	Exposes the Service using a cloud provider's load balancer .
ExternalName	Maps the Service to an external domain name instead of a Pod.

Creating a Service in Kubernetes

Services can be created using **kubectl** commands or **YAML configuration files**.



Method 1: Using **kubectl expose** Command

To expose a Deployment named **my-deployment** as a Service:

```
kubectl expose deployment my-deployment --type=ClusterIP --port=80
```

Verify the Service:

```
kubectl get services
```

To check detailed information about the Service:

```
kubectl describe service my-deployment
```

💡 This creates a **ClusterIP** Service, allowing internal access within the cluster.

Method 2: Creating a Service Using YAML

Using YAML provides more control over the Service configuration.

Step 1: Create a Service YAML File

Create a file named **service.yaml**:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
```



```
targetPort: 80
```

```
type: NodePort
```

Step 2: Apply the Service

Run the following command:

```
kubectl apply -f service.yaml
```

Step 3: Verify the Service

Check if the Service is running:

```
kubectl get services
```



Using YAML makes the configuration reusable and easy to modify.

Accessing a Service

Accessing a **ClusterIP** Service (Internal Only)

Get the Service IP using:

```
kubectl get service my-service
```

Access it from another Pod using:

```
curl http://my-service
```

Accessing a **NodePort** Service (External Access)

Find the NodePort:

```
kubectl get service my-service
```

Access the Service using:



```
curl http://<node-ip>:<node-port>
```

Accessing a **LoadBalancer** Service (Cloud Providers)

For cloud-based clusters (AWS, GCP, Azure), create a **LoadBalancer** Service:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Apply and check the external IP:

```
kubectl get service my-service
```

Deleting a Service

To delete a Service:

```
kubectl delete service my-service
```



Deleting a Service does not affect the running Pods.



Common Issues and Troubleshooting

Problem	Solution
Cannot access Service	Check Service type using <code>kubectl get services</code>
No external IP assigned	Ensure the correct <code>type</code> (NodePort/LoadBalancer) is used
<code>Connection refused</code> error	Ensure Pods are running and labeled correctly
No endpoints available	Check if Pods match the Service selector (<code>kubectl get endpoints</code>)

Know More (FAQs)

1. What is the difference between a Service and a Deployment?

- A **Deployment** manages Pods.
- A **Service** provides a stable way to access Pods.

2. How does a Service find the right Pods?

Services use **selectors** to match Labels on Pods.

selector:

```
app: my-app
```

3. Can a Service balance traffic between multiple Pods?

Yes! A Service automatically **distributes traffic** across all matching Pods.

4. What happens if a Pod crashes?

The Service automatically sends traffic to the remaining Pods.



5. Can I expose a Service to the internet?

Yes! Use **LoadBalancer** or **Ingress** for external access.

Conclusion

Services are **essential for networking in Kubernetes**. They provide a stable way to **connect, load balance, and expose applications** running inside a cluster.

👉 Try creating different types of Services and test how they work in your Kubernetes setup!